

WIG20 AI Agent

Autorzy: Jakub Zakrzewski Jan Zubalewicz

15 stycznia 2026

Spis treści

1	Streszczenie	4
2	Cel i Zakres Projektu	4
3	Funkcjonalnosc	4
3.1	Pobieranie Danych	4
3.2	Baza Danych	5
3.3	API REST	5
3.4	AI i Przetwarzanie Jezyka	5
4	Architektura Systemu	5
4.1	Komponenty	5
4.1.1	scraper.py - Pobieranie Danych	6
4.1.2	db_updater.py - Aktualizacja Bazy	6
4.1.3	main.py - REST API	6
4.1.4	mcp_server.py - Model Context Protocol	6
5	Technologie	7
5.1	Stack Techniczny	7
5.2	Wymagania Systemowe	7
6	Deployment i Infrastruktura	7
6.1	Docker Compose	7

6.2	Health Checks i Zaleznosci	8
6.3	Konfiguracja Zmiennych Srodowiskowych	8
7	Przepływ Danych i Architektura	8
7.1	Sekwencja Zdarzen	8
7.2	Diagram Przepływu	8
8	Interfejs API	9
8.1	Endpoint POST /ask	9
8.2	Przykłady Zapytan	9
8.2.1	Porownanie Spółek	9
8.2.2	Przegląd Rynku	10
8.2.3	Analiza Pojedynczej Spółki	10
9	Baza Danych	10
9.1	Schemat Tabel	10
9.1.1	Tabela Symbols	10
9.1.2	Tabela Prices	10
9.2	Operacje na Bazie	10
10	Bezpieczenstwo	11
10.1	Implementowane Mechanizmy	11
10.2	Zalecenia dla Produkcji	11
11	Instalacja i Urzadzanie	11
11.1	Kroki Instalacji	11
11.2	Komendy	11
11.3	Dostęp do Serwisów	12
12	Monitorowanie i Utrzymanie	12
12.1	Logowanie	12
12.2	Czyszczenie Bazy Danych	12

13 Przyszłe Rozszerzenia	13
13.1 Krotkoterminowe (1-2 miesiace)	13
13.2 Długotermionowe (3-6 miesiecy)	13
14 Struktura Projektu	13
15 Podsumowanie	13

1 Streszczenie

Projekt WIG20 Agent to zaawansowany system do automatycznego pobierania, przechowywania i analizy danych giełdowych z indeksu WIG20. Rozwiązanie integruje nowoczesne technologie web scrappingu, relacyjne bazy danych (MySQL) oraz model sztucznej inteligencji (Ollama) do odpowiadania na pytania użytkowników w języku naturalnym.

System realizuje kompleksowy przepływ danych zaczynający się od pobrania informacji z portalu finansowego Bankier.pl, poprzez przechowywanie w bazie danych, aż do prezentacji wyników użytkownikowi za pomocą inteligentnego agenta AI.

Główne możliwości:

- Automatyczne i okresowe pobieranie cen akcji ze strony Bankier.pl
- Przechowywanie danych w strukturyzowanej bazie MySQL z historią aktualizacji
- REST API do wysyłania zapytań i komunikacji z systemem
- Inteligentne odpowiedzi na pytania użytkowników za pomocą modelu Ollama
- Obsługa wielu języków (polski i angielski)

2 Cel i Zakres Projektu

Celem projektu jest stworzenie narzędzia do:

1. **Automatyzacji pobierania danych** - system periodycznie skanuje stronę Bankier.pl i ekstrahuje dane o cenach akcji spółek wchodzących w skład indeksu WIG20
2. **Przechowywania danych** - wszystkie zebrane dane są zapisywane w bazie MySQL z metadanymi czasowymi
3. **Udostępniania interfejsu** - REST API umożliwia komunikację z systemem poprzez zapytania HTTP
4. **Analizy inteligentnej** - model AI (Ollama) przetwarza pytania użytkowników i generuje odpowiedzi oparte na aktualnych danych

3 Funkcjonalność

System oferuje następujące funkcjonalności:

3.1 Pobieranie Danych

Automatyczne pobranie danych z Bankier.pl obejmuje:

- Symbole handlowe spółek (np. PKO, PKNORLEN, ORLEN)

- Ceny akcji wyrażone w PLN
- Kapitalizacje rynkowe spółek
- Znaczniki czasowe (timestamp) każdego pobrania

3.2 Baza Danych

MySQL przechowuje:

- Tabele **Symbols** - liste wszystkich spółek z ich kapitalizacja
- Tabele **Prices** - historie cen z datami aktualizacji
- Unikatowe indeksy zapobiegające duplikatom
- Relacje między tabelami poprzez klucze obce

3.3 API REST

Endpoint POST /ask umożliwia:

- Wysyłanie zapytań w języku naturalnym
- Klasyfikację intencji zapytania (porównanie, analiza, przegląd)
- Pobieranie odpowiednich danych z bazy
- Zwracanie wzbogaconych odpowiedzi

3.4 AI i Przetwarzanie Języka

Ollama z modelem Qwen2.5 odpowiada za:

- Rozumienie zapytań w języku polskim i angielskim
- Porównywanie spółek na podstawie danych
- Analizę trendów rynkowych
- Generowanie naturalnych odpowiedzi dla użytkownika

4 Architektura Systemu

4.1 Komponenty

System składa się z czterech głównych komponentów:

4.1.1 `scraper.py` - Pobieranie Danych

Modul odpowiedzialny za:

- Wysyłanie zapytan HTTP do Bankier.pl
- Parsowanie odpowiedzi HTML przy uzyciu BeautifulSoup
- Ekstrakcje danych za pomoca wyrazow regularnych (regex)
- Obsluge bledów i ponowne probowanie w przypadku niepowodzenia
- Emulacje przegladarki poprzez User-Agent (unikanie blokady)

4.1.2 `db_updater.py` - Aktualizacja Bazy

Warstwa bazodanowa zajmujaca sie:

- Tworzeniem tabel (Symbols, Prices) przy pierwszym uruchomieniu
- Zapisywaniem danych z scrapera do MySQL
- Operacjami UPSERT dla idempotentnosci
- Obsuga duplikatow poprzez unikatowe indeksy
- Logowaniem i raportowaniem bledów

4.1.3 `main.py` - REST API

FastAPI serwer dostarczajacy:

- Endpoint `POST /ask` do wysylania pytań
- Klasyfikacje typu zapytania (porownanie/analiza/przegląd)
- Logike pobierania danych z bazy na podstawie zapytania
- Integracje z modelem Ollama
- Dokumentacje API (Swagger UI na `/docs`)

4.1.4 `mcp_server.py` - Model Context Protocol

Zaawansowany serwer komunikacyjny oferujacy:

- Zasoby (resources): `stock://wig20/all`, `stock://wig20/prices`
- Narzedzia (tools): `set_price_alert`
- Transport Server-Sent Events (SSE) dla aktualizacji real-time
- Wsparcie dla zaawansowanych integracji

5 Technologie

5.1 Stack Techniczny

- **Python 3.11** - język programowania (wydajny, źródła open-source)
- **FastAPI** - nowoczesny framework do budowy REST API
- **Uvicorn** - szybki serwer ASGI
- **MySQL 8.0** - relacyjna baza danych
- **BeautifulSoup4** - parsing HTML
- **requests** - HTTP client dla Pythona
- **Pydantic** - walidacja danych i type hints
- **Ollama** - framework dla modeli LLM
- **Qwen2.5** - model AI/LLM
- **Docker Compose** - orkiestracja kontenerów

5.2 Wymagania Systemowe

- Python 3.11 lub nowszy
- Docker 20.10+
- Docker Compose 1.29+
- RAM: minimum 4GB (zalecane 8GB dla Ollama)
- Dysk: minimum 10GB na bazę danych i model AI
- Dostęp do internetu dla pobrania modelu Qwen2.5

6 Deployment i Infrastruktura

6.1 Docker Compose

System uruchamia się w czterech izolowanych kontenerach:

Nazwa	Obraz	Port	Opis
db	mysql:8.0	3306	Baza danych MySQL
ollama	ollama/ollama	11434	Model AI/LLM
api	local/wig20:latest	8000	FastAPI service
mcp	local/wig20:latest	-	MCP Server

Tabela 1: Serwisy Docker Compose

6.2 Health Checks i Zaleznosci

System implementuje:

- Health check dla MySQL: `mysqladmin ping -h localhost`
- Timeout: 20 sekund
- Liczba prob: 10 ponownych prob
- Zaleznosci: API i MCP czekaja na gotowosc bazy danych
- Uruchamianie w okreslonym porzadku dla spójności

6.3 Konfiguracja Zmiennych Srodowiskowych

Plik `.env` zawiera:

```
MYSQL_ROOT_PASSWORD=haslo_root
MYSQL_DATABASE=wig20
MYSQL_USER=agent
MYSQL_PASSWORD=haslo_agenta
```

7 Przepływ Danych i Architektura

7.1 Sekwencja Zdarzen

Pełny przepływ danych:

1. **Pobranie:** Scraper wysyla zapytanie do Bankier.pl
2. **Parsing:** BeautifulSoup ekstrahuje HTML i wydobywa dane
3. **Zapis:** `db_updater` zapisuje dane do MySQL z deduplicacją
4. **Zapytanie:** Uzytkownik wysyla POST na `/ask`
5. **Klasyfikacja:** API okresla typ zapytania
6. **Pobieranie:** Zapytania SQL pobieraja relewalntne dane
7. **AI:** Dane wysylane do Ollamy z promptem
8. **Odpowiedz:** Model generuje tekst i zwraca uzytkownikowi

7.2 Diagram Przeplywu

```
Bankier.pl
|
| HTML
v
```

```

Scraper (scraper.py)
|
| Symbole, Ceny
v
DB Updater (db_updater.py)
|
| SQL INSERT/UPDATE
v
MySQL Database
|
| SQL SELECT
v
FastAPI (main.py)
|
| Dane + Prompt
v
Ollama (Model AI)
|
| Tekst odpowiedzi
v
Uzytkownik/Client

```

8 Interfejs API

8.1 Endpoint POST /ask

Główny endpoint do komunikacji z systemem.

Request:

```

POST /ask HTTP/1.1
Content-Type: application/json

{
  "prompt": "Porównaj PKO i PKNORLEN"
}

```

Response:

```

{
  "response": "PKO Bank Polski handluje na poziomie 35.50 PLN,
  podczas gdy PKNORLEN na 34.20 PLN.
  PKO jest droższe o 1.30 PLN...",
  "data": {...}
}

```

8.2 Przykłady Zapytan

8.2.1 Porównanie Spółek

Zapytanie: "Porównaj PKO z PZU"

Odpowiedz: System pobiera ceny obu spółek z bazy i generuje porównanie oparte na danych.

8.2.2 Przegląd Rynku

Zapytanie: "Jaki jest stan rynku WIG20?"

Odpowiedz: Pobieranie wszystkich aktualnych cen i generowanie ogólnego przeglądu.

8.2.3 Analiza Pojedynczej Spółki

Zapytanie: "Analiza ALLEGRO"

Odpowiedz: Dane o spółce z historycznym kontekstem i analiza.

9 Baza Danych

9.1 Schemat Tabel

9.1.1 Tabela Symbols

Kolumna	Typ	Opis
id	INT PRIMARY KEY	Identyfikator unikatowy
name	VARCHAR(255) UNIQUE	Symbol spółki (PKO, PKNORLEN itp.)
market_cap	FLOAT	Kapitalizacja rynkowa

Tabela 2: Struktura tabeli Symbols

9.1.2 Tabela Prices

Kolumna	Typ	Opis
id	INT PRIMARY KEY	Identyfikator unikatowy
symbol_id	INT FOREIGN KEY	Odniesienie do Symbols
value	FLOAT	Cena akcji w PLN
time	DATETIME	Czas pobrania danych

Tabela 3: Struktura tabeli Prices

9.2 Operacje na Bazie

System wykorzystuje operacje:

- **INSERT ... ON DUPLICATE KEY UPDATE** - idempotentne zapisywanie
- **UNIQUE(symbol_id, time)** - unikanie duplikatów cen
- **FOREIGN KEY** - konsystencja danych
- **INDEX** - szybkie wyszukiwanie

10 Bezpieczeństwo

10.1 Implementowane Mechanizmy

- **Hasła w .env:** Dane wrażliwe nie są w kodzie
- **Health Checks:** Sprawdzenie gotowości serwisów
- **Timeout:** Limity czasowe na połączenia (10-20 sekund)
- **Error Handling:** Logowanie błędów z kontekstem
- **UNIQUE Constraints:** Zapobieganie duplikatom
- **User-Agent Emulation:** Unikanie blokady przez scraper

10.2 Zalecenia dla Produkcji

- Wdrożenie autentykacji API (JWT lub API keys)
- Dodanie rate limiting na endpoint /ask
- Szyfrowanie połączeń do MySQL (SSL/TLS)
- Regularne aktualizacje zależności Python
- Monitoring i logowanie akcji użytkowników
- Regularne backupy bazy danych
- Wdrożenie WAF (Web Application Firewall)

11 Instalacja i Urzadzanie

11.1 Kroki Instalacji

1. Klonowanie repozytorium z GitHub
2. Konfiguracja pliku .env z hasłami
3. Budowa kontenerów Docker
4. Uruchomienie Docker Compose
5. Pobranie modelu Ollama (qwen2.5)
6. Weryfikacja dostępności API na `http://localhost:8000`

11.2 Komendy

```
# Uruchomienie
docker-compose up -d

# Zatrzymanie
docker-compose down
```

```
# Logi API
docker logs wig20_api -f

# Logi MySQL
docker logs wig20_db -f

# Status kontenow
docker-compose ps
```

11.3 Dostęp do Serwisów

- **FastAPI:** <http://localhost:8000>
- **Swagger UI:** <http://localhost:8000/docs>
- **ReDoc:** <http://localhost:8000/redoc>
- **Ollama:** <http://localhost:11434>
- **MySQL:** localhost:3306 (uzytkownik: agent)

12 Monitorowanie i Utrzymanie

12.1 Logowanie

Wszystkie komponenty logują do stdout / stderr:

- API loguje zapytania, błędy, czasy odpowiedzi
- Scraper loguje status pobrania i błędy połączenia
- DB Updater loguje operacje SQL
- Ollama loguje inference requests

12.2 Czyszczenie Bazy Danych

Zalecane operacje:

```
-- Czyszczenie danych starszych niz 30 dni
DELETE FROM Prices
WHERE time < DATE_SUB(NOW(), INTERVAL 30 DAY);

-- Optymalizacja tabeli
OPTIMIZE TABLE Prices;
OPTIMIZE TABLE Symbols;

-- Sprawdzenie rozmiaru
SHOW TABLE STATUS LIKE 'Prices';
```

13 Przyszłe Rozszerzenia

13.1 Krotkoterminowe (1-2 miesiace)

- Alerty cenowe (email/SMS/push notification)
- Cache Redis dla czestych zapytan
- Filtry czasowe w /ask ("cena z ostatniego miesiaca")
- Wsparcie dla innych indeksów (WIG, mWIG40, sWIG80)
- Statystyki i wykresy cenowe

13.2 Dlugotermionowe (3-6 miesiecy)

- Machine Learning dla predykcji cen
- Frontend webowy (React/Vue.js)
- Aplikacja mobilna (iOS/Android)
- Integracja z platformami tradingowymi
- Raportowanie i eksport danych (CSV, PDF)
- Dashboard z wizualizacją danych
- Wsparcie dla portfeli uzytkowników

14 Struktura Projektu

```
wig20-agent/  
main.py           # FastAPI server  
scraper.py        # Web scraping  
db_updater.py     # Database operations  
mcp_server.py     # MCP protocol server  
client.py         # Client implementation  
requirements.txt  # Python dependencies  
Dockerfile        # Container image  
docker-compose.yml # Multi-container setup  
REPORT.md         # Markdown documentation  
RAPORT.tex        # LaTeX report  
.env.example      # Environment template  
.gitignore        # Git ignore rules  
__pycache__/_     # Python cache
```

15 Podsumowanie

WIG20 Agent to kompleksowy, skalowany system do inteligentnej analizy danych giełdowych. Architektura oparta na mikroservisach i konteneryzacji zapewnia łatwosc wdrozenia i utrzymania. Integracja z modelami AI umożliwia interaktywne odpowiadanie na pytania uzytkownikow w jezyku naturalnym.

System automatycznie:

- Pobiera i aktualizuje dane o cenach akcji
- Przechowuje dane w strukturyzowanej bazie
- Odpowiada na pytania użytkowników
- Dostarcza API REST do integracji

Projekt ma potencjał do dalszego rozwoju w kierunku zaawansowanej analizy finansowej, predykcji trendów i wsparcia dla decyzji inwestycyjnych.